CHAPTER 21

AI ENGINEERING ROADMAP · STUDY NOTES

Memory in Agentic AI

Why LLMs have no intrinsic memory, and how short-term and long-term memory are built around them from first principles — a framework-agnostic foundation for every agentic system.

WHAT THIS CHAPTER COVERS

- LLMs as stateless parametrized functions
- Context window and in-context learning
- Short-term memory via conversation history
- Thread-scoped memory in chatbots
- Three limitations of short-term memory
- Episodic, semantic & procedural long-term memory
- The 4-step long-term memory architecture
- Memory tools: LangMem, Mem0, Supermemory

Memory in Agentic AI

RECAP — WHY THIS CHAPTER EXISTS

Chapters 9 and 10 already showed *how* to wire short-term memory (checkpointers, threads) and persistence into a LangGraph chatbot. This chapter steps back and builds the entire concept of memory — short-term and long-term — from first principles, independent of any single framework. Treat it as the theoretical foundation underneath the LangGraph-specific implementation you already know.

1. LLMs Are Stateless by Design

Definition — The LLM as a Parametrized Math Function

At inference time, an LLM can be represented as a *parametrized mathematical function*:

```
y = f(x, theta)
```

A parametrized function is one whose output depends not only on its input, but also on a separate set of parameters. The classic example is linear regression, **y = mx + b**, where **m** and **b** are learned from data rather than supplied by the caller. An LLM follows the exact same shape — just at a vastly larger scale.

THE THREE COMPONENTS

- **theta** — billions of trained parameter values, fixed once training finishes. The user has zero control over theta at inference time.
- **x** — the input tokens sent during inference (the prompt). This is the only thing the user can change.
- **y** — the output tokens produced from combining the fixed theta with the user-supplied x.

Why It Exists — The Statelessness Property

This mathematical framing reveals an important property: the function is *stateless*. A system is stateless if its output depends only on the current input and not on anything that happened before. Concretely, if an LLM is called twice — producing y1 from x1, then y2 from x2 — the computation of y2 has no access to x1 or y1. theta is identical across both calls and never changes; only x changes, and each call is fully independent of the one before it.

IMPORTANT

Statelessness is not a bug to be patched — it is a structural property of how inference works. Every memory system built around an LLM is, at its core, a way of re-feeding past context back into x on each call, never a way of making the function itself remember.

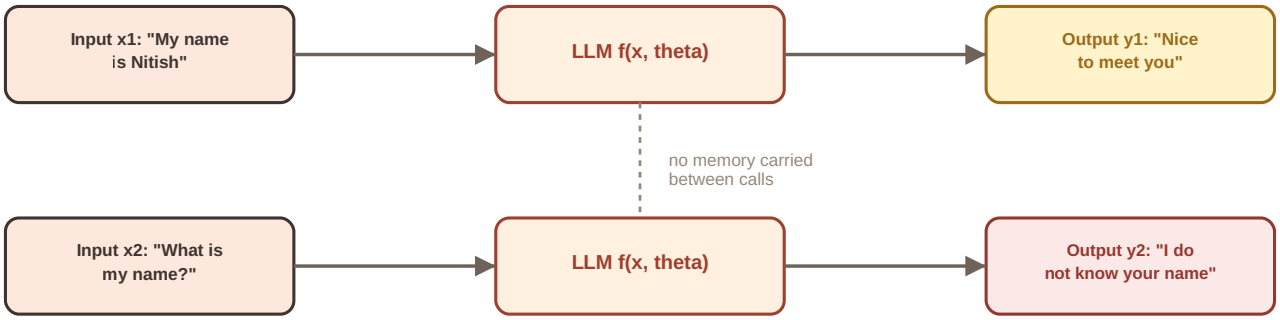
Example — Proving Statelessness in Code

Calling an LLM twice in isolation demonstrates the property directly:

```
# Call 1
response_1 = llm.invoke("My name is Nitish")
# -> "Nice to meet you, Nitish!"

# Call 2 (fresh call, same LLM, same parameters)
response_2 = llm.invoke("What is my name?")
# -> "I'm sorry, I do not know your name."
```

The second call cannot recall the first, even though both were made to the same model moments apart. Each invocation is unique and independent — the previous call left no trace.



theta (model parameters) is fixed and identical in both calls — only x changes, and y2 does not depend on x1 or y1.

Figure 21.1 — Two independent LLM calls: y2 depends only on x2 and theta, never on x1 or y1.

> ↻ **Quick Revision — Statelessness**
>
> - y = f(x, theta) represents an LLM at inference time.
> - theta (billions of parameters) is fixed at inference; only x (the prompt) can be changed by the user.
> - A stateless system's output depends only on the current input, never on past calls.
> - Each LLM invocation is independent — no call can see the input or output of a prior call.

2. The Core Problem: LLMs Have No Memory

Because LLMs are stateless at inference, a direct consequence follows: **LLMs have no intrinsic memory**. They cannot recall past conversations on their own — the capability simply does not exist inside the function.

This collides with a second fact: almost no real-world GenAI application can function without memory. A chatbot without memory forces the user to repeat context on every message, producing a highly frustrating experience. A naive implementation that calls the LLM fresh on every user message, without considering prior turns, will visibly fail to maintain continuity.

> **COMMON MISTAKE**
>
> Building a chat application that simply calls `llm.invoke(x)` on each new user message, without reattaching prior turns, is a frequent beginner mistake. The result looks functional for a single message but breaks down the moment the user expects the system to remember anything said earlier.

This creates a deadlock: LLMs have no memory of their own, yet virtually every GenAI application needs memory to be usable. The resolution is to build memory **externally**, around the LLM, rather than expecting the model itself to provide it.

> ↻ **Quick Revision — The Core Problem**
>
> • LLMs have no intrinsic memory because inference is stateless.
>
> • Virtually no GenAI or agentic application can function without memory.
>
> • The fix is architectural: build a memory system around the LLM, not inside it.

3. Context Window and In-Context Learning

Building memory externally relies on two foundational concepts.

Concept 1 — Context Window

Definition: the context window is the amount of text an LLM can read and remember at one time before producing an answer.

A useful analogy is a camera lens: a small lens captures a small part of the scene; a large lens captures much more. The context window plays the same role for an LLM — a larger window lets it process more text before answering. Modern LLMs typically offer context windows of 128K tokens or more; some models (e.g. Gemini) offer windows around 1 million tokens, roughly enough to hold around 200 pages of PDF content.

The key takeaway: a large context window means a substantial amount of text can be sent inside **x** on a single call.

Concept 2 — In-Context Learning

During training, an LLM absorbs patterns from huge volumes of data, and that knowledge becomes embedded in theta — this is called *parametric knowledge*. By default, the model answers a prompt by searching this parametric knowledge.

As LLMs grew larger, an emergent ability surfaced: the model can also use information present directly inside the prompt itself to answer a question, even if that information was never part of training. For example, a 100-page private company

PDF pasted into the prompt, followed by a question about it, can be answered correctly even though the model never saw that document during training.

Definition: in-context learning is an emergent ability that allows an LLM to use information and patterns present in the prompt itself, in addition to its trained parametric knowledge, to generate an answer.

> **REAL WORLD INSIGHT**
>
> In-context learning is the mechanism that makes RAG, memory, and most prompt-engineering techniques possible. Without it, pasting extra context into a prompt would simply add noise — it is precisely because LLMs can reason over prompt content as readily as over trained knowledge that external memory systems work at all.

> ↻ **Quick Revision — Context Window & In-Context Learning**
>
> - Context window: the maximum text an LLM can process before answering; modern models range from ~128K to ~1M tokens.
> - Parametric knowledge: facts absorbed into theta during training.
> - In-context learning: the ability to use prompt content directly, beyond parametric knowledge, to generate an answer.
> - Both concepts together make it possible to simulate memory by sending more context inside x.

4. Short-Term Memory Using Conversation History

How It Works

Combining a large context window with in-context learning gives a simple solution: on every new LLM call, concatenate the **entire conversation so far** into x, instead of sending only the latest message.

For a two-turn exchange, instead of computing y2 = f(x2, theta), the system computes:

```
y2 = f(concat(x1, y1, x2), theta)
```

The LLM reads x1 and y1 as part of its input for the second call, and because of in-context learning it can use that earlier exchange to answer x2 correctly (e.g. recalling a name mentioned earlier). The LLM itself has not gained memory — the system is simply providing continuity by re-sending the full conversation at every step.

Implementation

In code, this means maintaining a running list of messages and appending every new input and output to it before the next call:

```python
# Start a running list of messages
messages = ["My name is Nitish"]

# First call
response = llm.invoke(messages)
# -> "Nice to meet you, Nitish"
messages.append(response)   # append y1

# New user message arrives
messages.append("What is my name?")   # append x2

# Second call sends the full accumulated history
response = llm.invoke(messages)
# -> "Your name is Nitish"
```

The `messages` variable is now acting as **state**: before it existed, the system was stateless; because of it, the system has become *stateful*. This variable is commonly called a *conversation buffer*, since it dynamically accumulates the full conversation history.

> **IMPORTANT**
>
> If the process restarts (the script is re-run, the server resets), the `messages` variable is wiped and the LLM immediately forgets everything — it will no longer recognize the user. This is exactly why this form of memory is called **short-term memory**: it is temporary by construction.

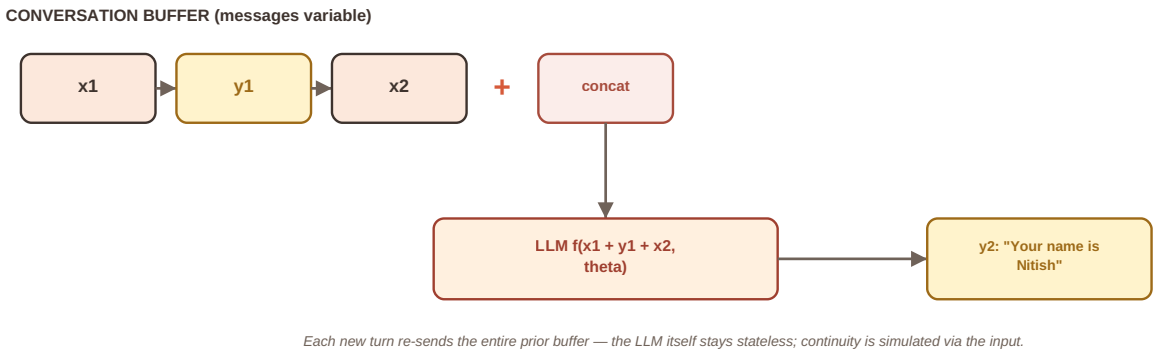


Figure 21.2 — Short-term memory as a growing conversation buffer re-sent on every call.

↻ Quick Revision — Short-Term Memory

- Short-term memory = resending the full conversation history as part of x on every call.
- The LLM stays stateless; the illusion of memory comes from the system re-providing context each turn.
- The running messages list is often called a conversation buffer.
- This memory is temporary: a process restart or reset erases it entirely.

5. How Chatbots Implement Short-Term Memory

Conversation / Thread Scoping

Real chatbots (ChatGPT, Gemini, and similar products) organize interactions into *conversations*: one session with the chatbot, from opening a topic to closing it. Each conversation gets its own short-term memory — the short-term memory is **conversation-scoped**. Outside its own conversation, that short-term memory does not exist.

In practice: only the messages belonging to the currently open conversation are concatenated into the buffer. The moment a user starts a new conversation, the buffer is first emptied, then repopulated only with the new conversation's messages.

Why It Is Scoped This Way

A single shared buffer across all of a user's conversations would not work at scale. With, say, 1,000 conversations, a single combined buffer would become far too long and incoherent for the LLM to process meaningfully — so a logical boundary is created instead: one conversation is one boundary, and short-term memory exists only inside that boundary.

Terminology — Threads

In many frameworks (including LangGraph), a conversation is called a *thread*. "Thread" and "conversation" are used interchangeably, which is why short-term memory is commonly described as **thread-scoped**: its scope is a single thread, and it does not exist outside that thread.

> **INTERVIEW TIP**
>
> If asked to define short-term memory precisely, lead with the scoping property: short-term memory is conversation-scoped (equivalently, thread-scoped). This single fact explains both why it is useful (clean continuity within a topic) and why it has limitations (no continuity across topics) — covered next.

↻ Quick Revision — Chatbot Scoping

- Each conversation gets its own independent short-term memory buffer.
- Switching to a new conversation empties the buffer and repopulates it only with the new conversation's messages.
- Scoping to one conversation avoids an unbounded, incoherent combined buffer across all conversations.
- 'Thread' is the common term for a conversation; short-term memory is described as thread-scoped.

6. Limitations of Short-Term Memory

Short-term memory, as implemented so far, has three significant limitations.

1. IT IS FRAGILE

Because the conversation buffer typically lives only in process memory (a variable), a code reset, server crash, or simply navigating to a new chat and back can wipe it. If a user switches to a new conversation and later returns to the previous

one without that history having been persisted anywhere, the earlier context is gone — the LLM treats the next message in that old conversation as entirely new.

> **BEST PRACTICE**
>
> Solve fragility with **persistence**: connect the short-term memory buffer to a database. Create a thread ID for each conversation; when switching away from a conversation, store its current messages against that thread ID; when returning, reload the messages for that thread ID back into the buffer before continuing.

2. THE CONTEXT WINDOW PROBLEM

Since every new call resends the entire conversation history, a long-running chat accumulates an ever-growing buffer. Eventually the token count of that buffer can exceed the LLM's context window. When this happens, the model can start producing incoherent replies or hallucinating, because it can no longer properly process everything it has been handed.

The most widely used fix combines two approaches:

- **Trimming** — instead of sending the entire history (e.g. 500 messages), send only the most recent N (e.g. the last 50), since the most relevant context is usually recent.
- **Summarization** — rather than discarding older messages outright, pass them to another LLM call to generate a compact summary, then send the recent N messages plus that summary together. The combined token count stays comfortably within the context window while still preserving older context in condensed form.

3. THREAD-SCOPED MEMORY BLOCKS PERSONALIZATION

Being scoped to a single thread is exactly what makes short-term memory useful within a conversation — but it also creates a serious drawback across conversations. Three concrete consequences:

- **No continuity between conversations.** If a user states a preference (e.g. "I prefer Python over any other language") in one conversation, that preference is forgotten the moment a new conversation starts — the assistant may go back to generating code in a different language entirely.
- **Learning never compounds over time.** Effort spent guiding the model toward a better solution (e.g. refining a SQL query from a subquery into an optimized window function) has to be repeated from scratch in every new conversation, because none of that refinement carries forward.
- **Cross-thread reasoning is impossible.** A user cannot ask "what did we discuss yesterday?" or "what solution did we land on last time this came up?" — the assistant has no access to any conversation other than the current one.

> **IMPORTANT**
>
> The root cause of all three problems is the same: short-term memory is locked inside a single conversation boundary by definition. A true personal assistant needs to build up a profile of the user across many conversations — which is, by definition, not possible with short-term memory alone.

↻ Quick Revision — Limitations

- Fragility: an in-memory buffer can be lost on reset/crash/switch; fix with a database + thread ID persistence pattern.
- Context window overflow: long chats can exceed the model's token limit, causing incoherence or hallucination.
- Fix for overflow: combine trimming (keep the most recent N messages) with summarization (compress older messages).
- Thread scoping blocks cross-conversation continuity, compounding learning, and cross-thread reasoning.
- These limitations motivate a fundamentally different kind of memory: long-term memory.

7. Why We Need Long-Term Memory

Solving the problems above requires a completely new kind of memory, with two essential properties.

PROPERTY 1 — SURVIVES BEYOND A SINGLE CONVERSATION

This new memory must store a special category of information: information that remains useful across sessions, potentially for days, weeks, or longer. For example, while writing a book over several months, relevant context from today's conversation with an assistant should still matter in a conversation held weeks later.

The kinds of information this memory needs to capture include:

- Who the user is — e.g. "the user is male, Indian, a YouTuber, a teacher who teaches AI."
- How the system/assistant should behave for this particular user.
- What has worked and what has failed for this user in the past — past decisions, past actions, and their outcomes.

PROPERTY 2 — HIGHLY SELECTIVE

This memory cannot simply absorb an entire chat transcript. It must extract only the most important, reusable pieces of information from each conversation — information that is stable, useful, and reusable over a long period — and discard everything else.

A memory with both properties is called **long-term memory**. Unlike short-term memory, which is destroyed the moment its conversation ends, long-term memory exists outside any single conversation, for an extended period of time.

↻ Quick Revision — Why Long-Term Memory

- Long-term memory must survive beyond a single conversation, sometimes for days or months.
- It must be highly selective, storing only stable, reusable, high-value information, not entire transcripts.
- It captures who the user is, how the system should behave for them, and what has worked or failed before.
- Unlike short-term memory, long-term memory exists independently of any single conversation's lifetime.

8. Types of Long-Term Memory

Long-term memory stores three distinct categories of information.

1. EPISODIC MEMORY

Captures what happened in the past — e.g. the user rejected a particular solution last session, a deployment used the wrong credentials, or a given approach succeeded (or failed) previously. Knowing this lets the current conversation answer questions like "what did we do last time?" or "have we already tried this?", letting the agent act more intelligently in the present.

2. SEMANTIC MEMORY

The most common and most important type. Composed of **facts** about the user and the system — e.g. "the user prefers Python," "the user is a beginner in software," "the system uses PostgreSQL," or "the budget constraint for this ticket-booking task is ₹10,000." This is essentially user-level or system-level factual memory.

3. PROCEDURAL MEMORY

Captures **how to do things** — strategies, rules, and learned behaviors. For example: "this user wants SQL problems solved without subqueries," "if tool X fails, always retry with tool Y," or "this user always wants step-by-step explanations." This type of memory compounds over time and is the main reason an agent starts to feel more tailored and comfortable to work with the longer it is used.

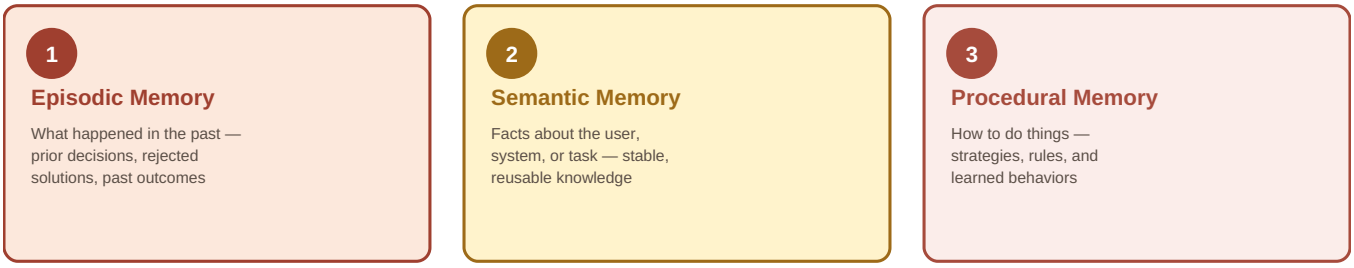


Figure 21.3 — The three categories of long-term memory.

REAL WORLD INSIGHT

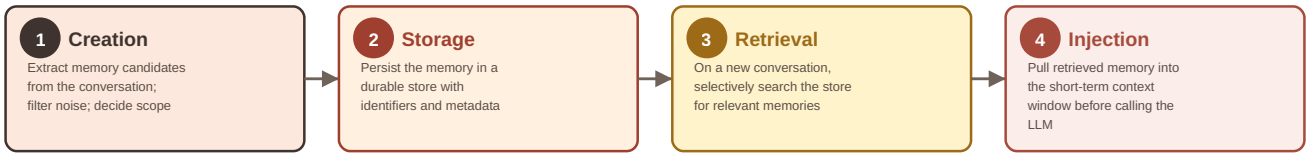
Semantic memory tends to be the easiest to implement and the highest-leverage starting point for most products, since user preferences and system facts are relatively stable and simple to extract. Episodic and procedural memory require more careful judgment about what is worth remembering and typically mature later in a product's memory system.

↻ Quick Revision — Types of Long-Term Memory

- Episodic memory: what happened in the past (past decisions, outcomes, rejected solutions).
- Semantic memory: facts about the user, system, or task — the most common and important type.
- Procedural memory: how to do things — strategies, rules, and learned behaviors.
- Together, these three types let an agent personalize and improve its behavior over time.

9. How Long-Term Memory Works

At a high level, a long-term memory system operates through four steps: **Creation, Storage, Retrieval,** and **Injection**.



Long-term memory never talks to the LLM directly — it is injected into short-term memory (the context window) first.

Figure 21.4 — The four-step long-term memory pipeline.

Step 1 — Creation (Update)

During an ongoing conversation, the system must determine whether anything that just happened is worth remembering beyond this conversation. It examines user messages, model responses, and (in tool-using systems) tool outputs, and performs four tasks:

- Extract memory candidates — e.g. the user says "I prefer Python over Java" becomes a candidate.
- Filter out noise — extract only the core fact, discarding irrelevant surrounding chatter.
- Decide scope — tag whether the memory is user-level, app-level, or agent-level.
- Decide the final action — create a new memory, update an existing one, or ignore the candidate entirely.

Step 2 — Storage

Once a piece of information is confirmed as a memory, it must be saved somewhere durable, since it needs to be reusable in the future. Storage involves two tasks: saving the memory in a durable store, and tagging it with identifiers and metadata so it can be retrieved efficiently later. The goal is that this memory survives restarts and crashes.

The choice of store depends on the type of memory being stored:

Memory Type	Typical Store
Semantic memory (facts)	Relational database or key-value store
Episodic memory (events/logs)	Log files or a document/text store
Semantic search needs	Vector database

Step 3 — Retrieval

When a user starts a new conversation, before replying the model (or the surrounding system) must decide: given the current situation, what should I remember right now? Retrieval performs: examining the current user input, deciding whether memory is needed at all, searching the memory store if so, and extracting only a small, relevant subset of memory — not everything in the store.

IMPORTANT

Retrieval is **selective, not exhaustive**. Unlike short-term memory (which sends the entire conversation buffer), long-term memory retrieval deliberately pulls back only the relevant subset of stored memories for the current situation.

Step 4 — Injection

The LLM never interacts with long-term memory directly. Retrieved memory is pulled into short-term memory — it becomes part of the context window / conversation buffer, and is injected alongside the user's own prompt before being sent to the model. From the model's perspective, injected long-term memory is indistinguishable from any other input tokens.

↻ Quick Revision — The 4-Step Architecture

- Creation: extract candidates, filter noise, decide scope, then create/update/ignore.
- Storage: persist durably with identifiers/metadata; store choice depends on memory type (relational DB, log/text store, or vector DB).
- Retrieval: selective, not exhaustive — search the store and pull only a relevant subset.
- Injection: long-term memory is folded into short-term memory (the context window) before reaching the LLM; it never talks to the LLM directly.

10. Challenges and Tools for Memory Systems

Three Core Challenges

- **Memory creation is hard.** Deciding which parts of a noisy, everyday conversation genuinely deserve to become long-term memory — and which will still matter later — is a difficult judgment call.
- **Relevant retrieval is hard.** Even with memory correctly created and stored, figuring out in real time which stored memories are actually relevant to the current conversation is its own challenging task.
- **Orchestrating the whole system is hard.** Building an already-complex agentic system and then layering a full memory pipeline (creation, storage, retrieval, injection) on top significantly increases engineering complexity, including wiring multiple memory stores into the system correctly.

Managed Memory Tools

A growing set of libraries and managed platforms now offer memory layers for GenAI and agentic applications, so that application builders do not need to implement the full creation/storage/retrieval pipeline themselves:

LangMem Mem0 Supermemory

Tool	Notes

LangMem	From the LangChain family; integrates easily with LangGraph for building AI agents.
Mem0	A popular managed memory-layer platform for GenAI applications.
Supermemory	A newer managed memory platform for GenAI apps, notable for rapid recent growth.

REAL WORLD INSIGHT

The emergence of well-funded managed memory platforms signals that memory is being treated as its own infrastructure layer for agentic AI — similar to how vector databases became a distinct layer for RAG. Expect this space to keep growing quickly.

The Future of Memory in LLMs

All of the external memory engineering discussed in this chapter exists because LLMs have no intrinsic memory of their own. Research is actively exploring LLM architectures with built-in memory, reducing the need for this much external scaffolding. One example referenced in the lecture is Google's research on **Titans** and a follow-up called **MIRAGE**, which explore transformer-style architectures designed to carry intrinsic memory. This remains an active research area rather than a production-ready replacement for the systems covered in this chapter.

↻ Quick Revision — Challenges & Tools

- Three core challenges: deciding what to remember, retrieving what's relevant, and orchestrating the full system.
- Managed memory tools: LangMem (LangChain family, integrates with LangGraph), Mem0, Supermemory.
- These tools let builders focus on the application while the memory layer is handled for them.
- Research (e.g. Google's Titans / MIRAGE) is exploring LLM architectures with intrinsic memory, an active but not yet production-standard direction.

K. Key Takeaways

- **Statelessness:** an LLM at inference is a parametrized function y = f(x, theta); theta is fixed, so each call is independent of every prior call.
- **The core problem:** LLMs have no intrinsic memory, yet nearly every GenAI application needs memory — so memory must be built externally.
- **Context window + in-context learning:** a large token budget plus the ability to reason over prompt content are what make external memory systems possible at all.
- **Short-term memory:** resending the full conversation buffer on every call; stateful but temporary and conversation/thread-scoped.
- **Short-term memory's limitations:** fragility (fix: persistence + thread IDs), context-window overflow (fix: trimming + summarization), and no continuity across conversations.

- **Long-term memory:** survives beyond a single conversation and is highly selective about what it stores.
- **Three types of long-term memory:** episodic (past events), semantic (facts), procedural (how-to strategies).
- **Four-step architecture:** Creation → Storage → Retrieval → Injection; long-term memory always reaches the LLM via short-term memory, never directly.
- **Tooling:** LangMem, Mem0, and Supermemory now provide managed memory layers so teams don't have to build this pipeline from scratch.

R. Revision Sheet

Stateless LLM

y = f(x, theta); theta is fixed at inference, so each call is fully independent of prior calls.

Context Window

The maximum amount of text an LLM can process before answering; modern models span roughly 128K to ~1M tokens.

In-Context Learning

The emergent ability of an LLM to use information present in the prompt itself, beyond its trained parametric knowledge.

Short-Term Memory

A conversation buffer re-sent in full on every LLM call; conversation/thread-scoped and lost on reset.

Thread Scoping

Each conversation (thread) has its own isolated short-term memory; it does not exist outside that thread.

Trimming + Summarization

Send only the most recent N messages plus a compressed summary of older ones to stay within the context window.

Long-Term Memory

Selective, durable memory that survives beyond a single conversation, capturing user/system-level information.

Episodic Memory

What happened in the past — prior decisions, outcomes, and rejected solutions.

Semantic Memory

Stable facts about the user, system, or task; the most common type of long-term memory.

Procedural Memory

Strategies, rules, and learned behaviors describing how to do things for a given user/system.

Creation & Storage

Extract/filter/scope candidate memories, then persist them durably with identifiers and metadata.

Retrieval & Injection

Selectively search long-term memory, then inject relevant results into short-term memory before the LLM call.

I. Interview Questions

Q1. Why is an LLM considered stateless at inference time, and how does the equation y = f(x, theta) illustrate this?

Q2. Explain the difference between parametric knowledge and in-context learning, and why in-context learning matters for building memory systems.

Q3. How is short-term memory typically implemented in a chatbot, and what role does the conversation buffer play?

Q4. What does it mean for short-term memory to be "thread-scoped," and why is it designed that way instead of using one global buffer?

Q5. Describe the three major limitations of short-term memory and one practical fix for each.

Q6. What two properties must a long-term memory system have, and why does short-term memory fail to provide them?

Q7. Differentiate between episodic, semantic, and procedural memory with an example of each.

Q8. Walk through the four steps of the long-term memory architecture: Creation, Storage, Retrieval, Injection.

Q9. Why does retrieval in long-term memory need to be "selective, not exhaustive," and what would happen if it weren't?

Q10. Why does long-term memory get injected into short-term memory rather than being sent to the LLM directly?

F. Further Reading

- LangMem documentation (LangChain ecosystem memory library for agents)
- Mem0 documentation (managed memory layer for GenAI applications)
- Supermemory documentation (managed memory platform for GenAI applications)
- LangGraph documentation — Persistence and Memory (checkpointer-based short-term memory, covered in Chapters 9–10 of this series)
- Google Research — "Titans" paper (intrinsic-memory transformer architecture research)